4. which optimization was the most difficult?

→ Coordinating the heartbeat monitor with the thread pool's capacity check — making sure a client being cleaned up for inactivity didn't get double-counted against the connection limit, and that reconnect attempts didn't pile up against a socket that was intentionally closing during ~~shut~~ shutdown.

5. what additional improvements are required to support 100 users?

→ A large dynamically-sized thread pool or move to asynchronous I/O instead of one thread per client, a load balancer or multiple server instances behind it, and persisting chat history/user data in a real database instead of flat JSON/csv files to avoid lock contention at higher concurrency.

instead of configuration

3. What reliability issue did you discover?

→ The security log showed clients being dropped by heart-beat timeout, handler exceptions and abrupt server shutdowns - all of which needed explicit handling (timeouts, graceful shutdown, reconnection) rather than assuming the network connection would just stay up. Wireshark also revealed that authentication is sent in plaintext, which is a reliability / security limitation carried over from the original protocol.

Assignment - 8

Handwritten Reflection Questions & Answers

1. Which optimization produced the greatest improvement?

→ The heartbeat mechanism paired with automatic reconnections since it change reliability from something we couldn't measure at all (no latency data existed before) to something concrete and monitored - 62 real round-trip samples averaging 1.5ms after the change.

2. Why is scalability important?

→ A chat server that only works for 2-3 users has no real value. scalability determines whether the same design can serve 10, 100 or 1000 users without redesigning it from scratch. Without it, every growth in users becomes a rewrite instead of configuration change.